Project Implementation Report: Detection and Remediation of CVE-1999-0678

Author: Pham Huu Minh - 23BI14302

Introduction

This report presents a comprehensive implementation of an Intrusion Detection System (IDS) for detecting and remediating the CVE-1999-0678 vulnerability. The project demonstrates a complete security testing workflow, from vulnerability introduction through automated and manual detection, IDS implementation, and final remediation. The testing environment utilized two virtual machines: an attacker machine running Kali Linux and a victim machine running Ubuntu Server.

Phase 1: Network Configuration and Connectivity

To establish a controlled test environment, two virtual machines were configured: an attacker machine (Kali Linux) and a victim machine (Ubuntu Server).

Network Verification

The network configuration was verified by checking IP addresses on both machines using the `ip a` command. Connectivity between the machines was established and confirmed using ping tests.

Action: Verified IP addresses and established connectivity between both machines using ping.

Results:

- Ubuntu Server IP Address: 192.168.45.132
- Kali Linux IP Address: 192.168.45.128
- Successful communication confirmed between Kali (192.168.45.128) and Ubuntu (192.168.45.132)

```
        ErrorLog ${APACHE_LOG_DIR}/error.log
        CustomLog ${APACHE_LOG_DIR}/access.log combined

        # For most configuration files from conf-available/, which are
        # enabled or disabled at a global level, it is possible to
        # include a line for only one particular virtual host. For example the
        # following line enables the CGI configuration for this host only
        # after it has been globally disabled with "a2disconf".
        #Include conf-available/serve-cgi-bin.conf
        Alias /doc "/usr/share/doc/"

        <Directory "/usr/share/doc/">

        Options +Indexes +MultiViews +FollowSymLinks

            AllowOverride None

           Require all granted

        </Directory>
</VirtualHost>



lto@lto:~$ sudo systemctl restart apache2
lto@lto:~$
lto@lto:~$
lto@lto:~$
lto@lto:~$
lto@lto:~$ sudo systemctl restart apache2
lto@lto:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
       valid_lft forever preferred_lft forever
2: ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:e1:55:b4 brd ff:ff:ff:ff:ff:ff
    altname enp2s1
    inet 192.168.45.132/24 metric 100 brd 192.168.45.255 scope global dynamic ens33
       valid_lft 1551sec preferred_lft 1551sec
    inet6 fe80::20c:29ff:fee1:55b4/64 scope link
       valid_lft forever preferred_lft forever
lto@lto:~$
```

Figure 1: Ubuntu Server IP address verification using `ip a` command showing interface ens33 with IP 192.168.45.132

```
lto@lto:~$ ping 192.168.45.128
PING 192.168.45.128 (192.168.45.128) 56(84) bytes of data.
64 bytes from 192.168.45.128: icmp_seq=1 ttl=64 time=0.530 ms
64 bytes from 192.168.45.128: icmp_seq=2 ttl=64 time=0.886 ms
64 bytes from 192.168.45.128: icmp_seq=3 ttl=64 time=0.727 ms
^C
--- 192.168.45.128 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2003ms
rtt min/avg/max/mdev = 0.530/0.714/0.886/0.145 ms
lto@lto:~$ _
```

Figure 2: Ping test from Ubuntu Server to Kali Linux (192.168.45.128) confirming successful connectivity with 0% packet loss

```
Session  Actions  Edit  View  Help
┌──(kali㉿lto)-[~]
└─$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
       valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:8f:f2:51 brd ff:ff:ff:ff:ff:ff
    inet 192.168.45.128/24 brd 192.168.45.255 scope global dynamic eth0
       valid_lft 1112sec preferred_lft 1112sec
    inet6 fe80::20c:29ff:fe8f:f251/64 scope link proto kernel_ll
       valid_lft forever preferred_lft forever
3: eth1: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:8f:f2:5b brd ff:ff:ff:ff:ff:ff
    inet 192.168.50.128/24 brd 192.168.50.255 scope global dynamic noprefixroute eth1
       valid_lft 1700sec preferred_lft 1700sec
    inet6 fe80::10d9:1c08:f5f0:a8ac/64 scope link noprefixroute
       valid_lft forever preferred_lft forever
4: docker0: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc noqueue state DOWN group default
    link/ether 02:42:d7:ea:e0:46 brd ff:ff:ff:ff:ff:ff
    inet 172.17.0.1/16 brd 172.17.255.255 scope global docker0
       valid_lft forever preferred_lft forever

┌──(kali㉿lto)-[~]
└─$ ping 192.168.45.132
PING 192.168.45.132 (192.168.45.132) 56(84) bytes of data.
64 bytes from 192.168.45.132: icmp_seq=1 ttl=64 time=0.396 ms
64 bytes from 192.168.45.132: icmp_seq=2 ttl=64 time=0.322 ms
64 bytes from 192.168.45.132: icmp_seq=3 ttl=64 time=0.396 ms
^C
--- 192.168.45.132 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2026ms
rtt min/avg/max/mdev = 0.322/0.371/0.396/0.034 ms

┌──(kali㉿lto)-[~]
└─$
```

Figure 3: Kali Linux IP address verification and ping test to Ubuntu Server (192.168.45.132) confirming bidirectional connectivity

System Update and Vulnerability Introduction

Prior to introducing the vulnerability, the Ubuntu server was updated to ensure all packages were current. The CVE-1999-0678 vulnerability was then intentionally introduced on the Ubuntu server to simulate a real-world security scenario.

System Update: The Ubuntu system was updated using `sudo apt update` to refresh package lists and ensure the Apache web server was properly installed.

```
lto@lto:~$ sudo apt update
Hit:1 http://vn.archive.ubuntu.com/ubuntu noble InRelease
Hit:2 http://vn.archive.ubuntu.com/ubuntu noble-updates InRelease
Hit:3 http://vn.archive.ubuntu.com/ubuntu noble-backports InRelease
Hit:4 http://security.ubuntu.com/ubuntu noble-security InRelease
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
44 packages can be upgraded. Run 'apt list --upgradable' to see them.
lto@lto:~$ sudo apt install apache2 -y
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
apache2 is already the newest version (2.4.58-1ubuntu8.8).
0 upgraded, 0 newly installed, 0 to remove and 44 not upgraded.
lto@lto:~$ _
```

Figure 4: Ubuntu system update and Apache2 installation verification

Environment Setup: The CVE-1999-0678 vulnerability was intentionally introduced on the Ubuntu server by modifying the `/etc/apache2/sites-available/000-default.conf` file to include an insecure `/doc` alias.

Configuration Change: The configuration file was modified to include the following insecure alias:

```
Alias /doc "/usr/share/doc/"
<Directory "/usr/share/doc/">
    Options +Indexes +MultiViews +FollowSymLinks
    AllowOverride None
    Require all granted
</Directory>
```

This configuration allows unauthorized access to system documentation files through directory traversal, which can expose sensitive information about the server's installed packages and system structure.

Figure 5: Modified Apache configuration file showing the vulnerable /doc alias configuration with directory indexing enabled

Phase 2: Vulnerability Scanning with Greenbone Vulnerability Manager (GVM)

The Greenbone Vulnerability Manager (GVM) was deployed on the Kali Linux machine to perform automated vulnerability scanning of the victim server.

GVM Installation and Configuration

GVM was installed and configured on the Kali Linux machine following standard installation procedures. The installation process included setting up the PostgreSQL database, downloading vulnerability feeds, and configuring the OpenVAS scanner.

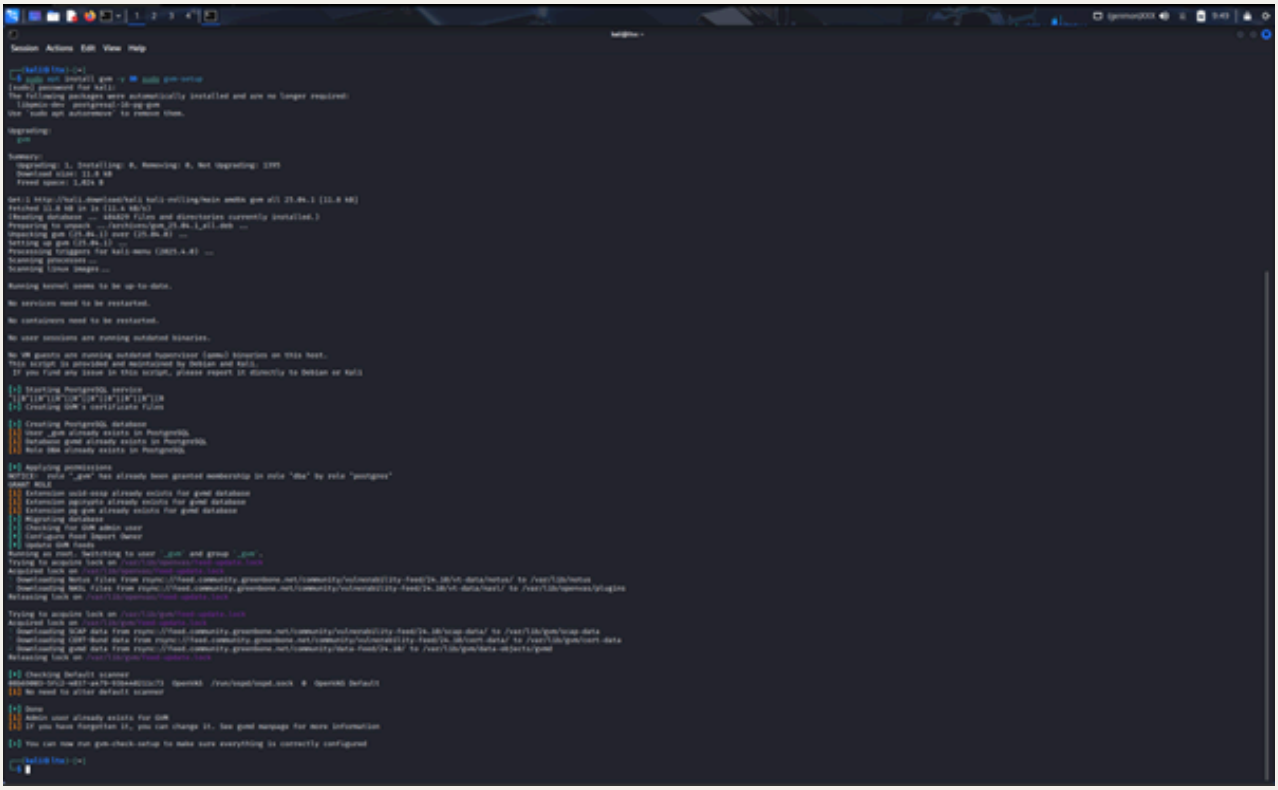

Figure 6: GVM installation process on Kali Linux showing package installation, database setup, and vulnerability feed downloads

Scan Execution

Action: Initiated a “Full and Fast” scan targeting the Ubuntu IP (192.168.45.132). The scan was configured to check all potential threats and vulnerabilities across all available ports and services using the Task Wizard.

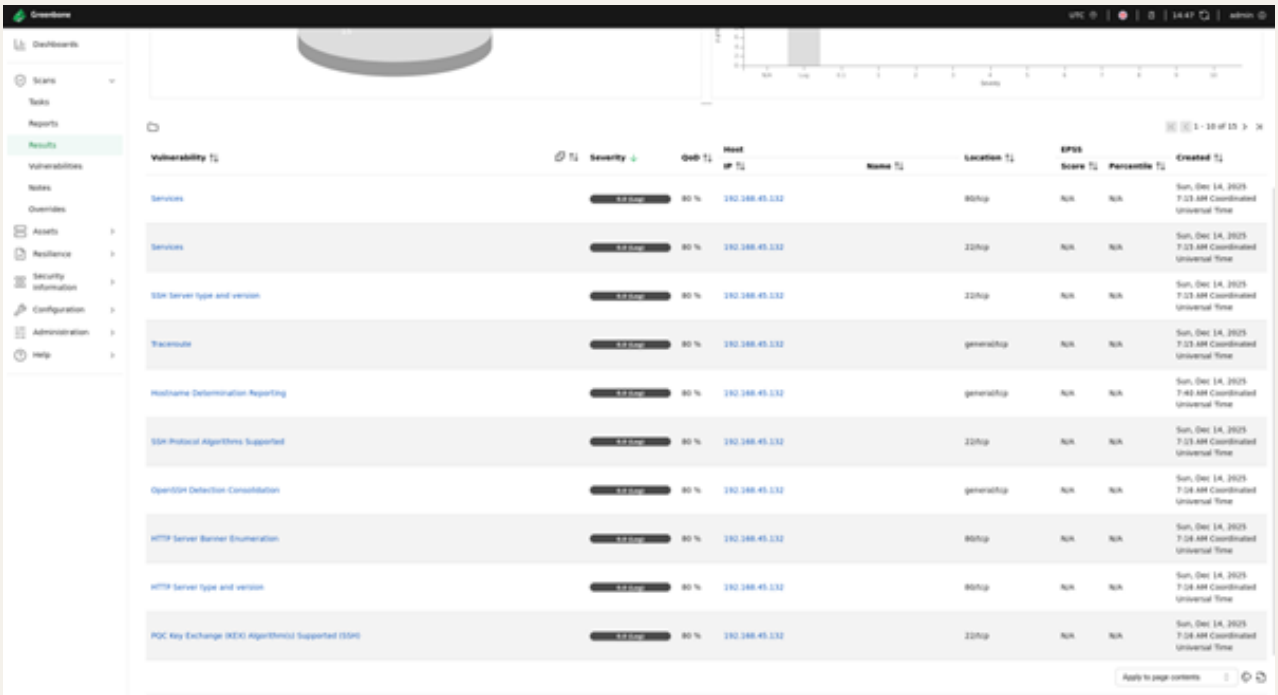

Figure 7: GVM Task Wizard configuration targeting Ubuntu server IP 192.168.45.132 with Full and Fast scan profile

Scan Results

Observation: The scanner successfully identified the service “HTTP Server type and version” on TCP Port 80.

Findings:

- Service detected: HTTP Server type and version
- Port: 80/tcp
- Severity classification: Log level (informational)

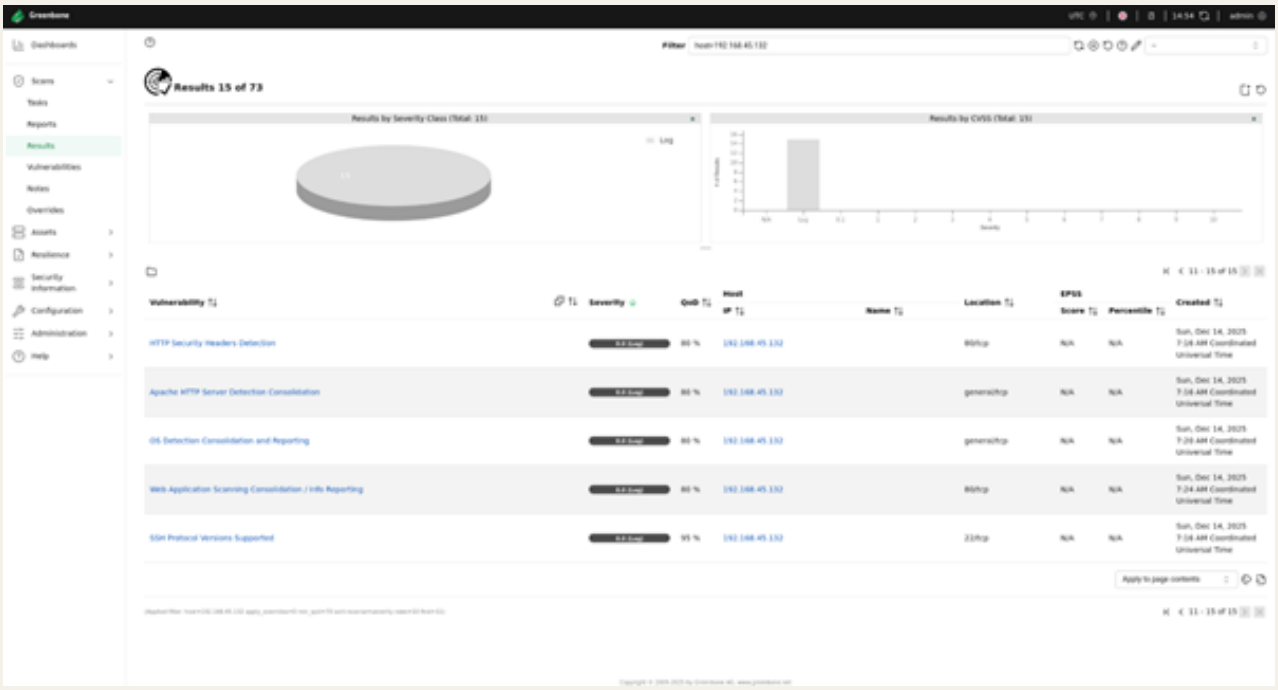

Figure 8: GVM scan results showing “HTTP Server type and version” detection on Port 80/tcp with Log severity classification and detailed vulnerability distribution

Analysis: While the automated scanner detected the web server, it classified the finding as “Log” level severity. To confirm the actual risk, manual verification was necessary. This demonstrates a limitation of automated scanning tools, which may not always recognize configuration-based vulnerabilities that require manual verification.

Phase 3: Manual Verification and Exploitation

Due to the scanner’s low severity rating, manual verification was performed to prove the vulnerability was active and dangerous.

Browser-Based Verification

Browser Test: The vulnerability was verified by accessing the exposed directory through a web browser. Firefox on the Kali Linux machine was used to navigate to `http://192.168.45.132/doc/`.

Result: The server returned a full directory listing, confirming unauthorized access to system documentation files. This directory listing exposed information about installed packages, system documentation, and potentially sensitive configuration details.

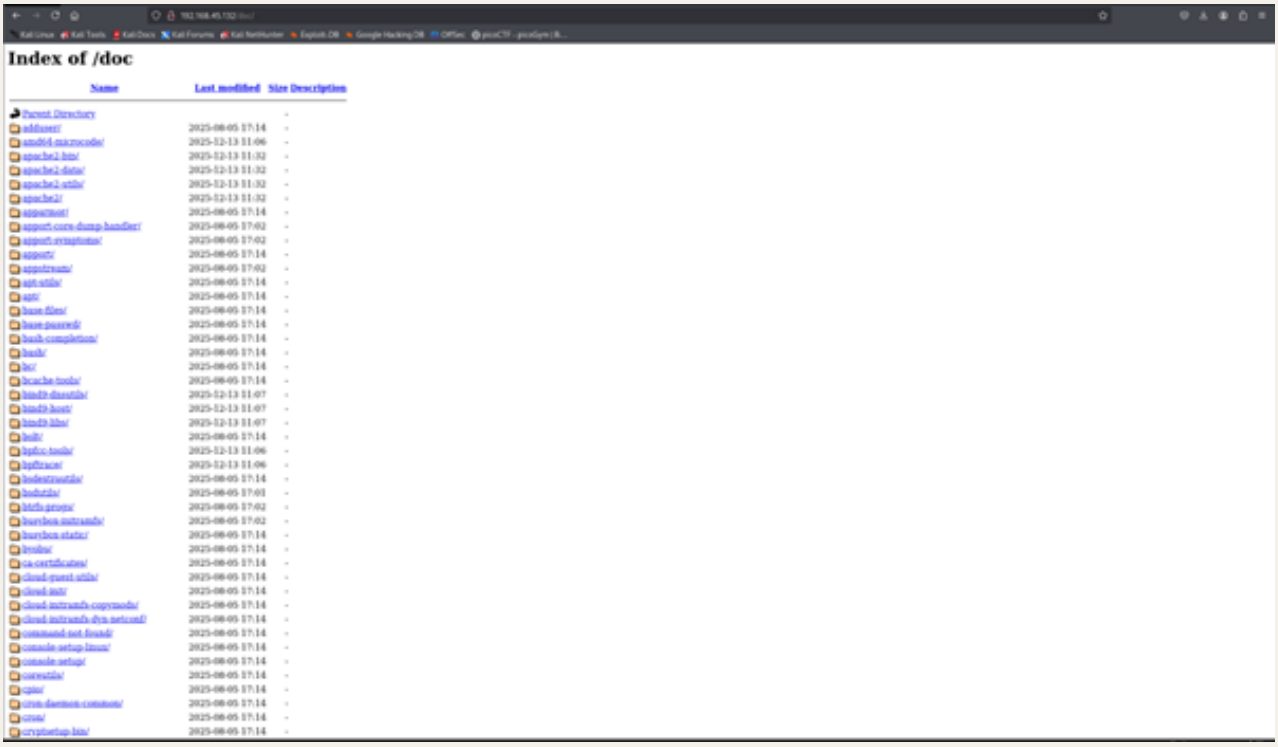

Figure 9: Firefox browser showing directory listing at /doc/ endpoint displaying system documentation files and directories

Command-Line Verification

Terminal Test: To further validate the vulnerability, a command-line test was performed using `curl`:

```
curl -I http://192.168.45.132/doc/
```

Result: The server returned a 200 OK status code, confirming the directory was accessible without authentication and that directory indexing was enabled.

Figure 10: curl command output showing HTTP/1.1 200 OK response before remediation, confirming directory accessibility

Risk Assessment

The manual verification confirmed that the vulnerability was active and exploitable. The exposed directory listing could be used by attackers to:

- Gather information about installed software and versions
- Identify potential additional vulnerabilities
- Map the system structure for further reconnaissance
- Access sensitive documentation files

Phase 4: Intrusion Detection System Implementation (Snort)

Snort, an open-source network intrusion detection system, was implemented on the Ubuntu server to detect exploitation attempts in real-time.

Snort Installation

Snort was installed on the Ubuntu server using the standard package management system. The installation included the Snort daemon, rule sets, and configuration files.

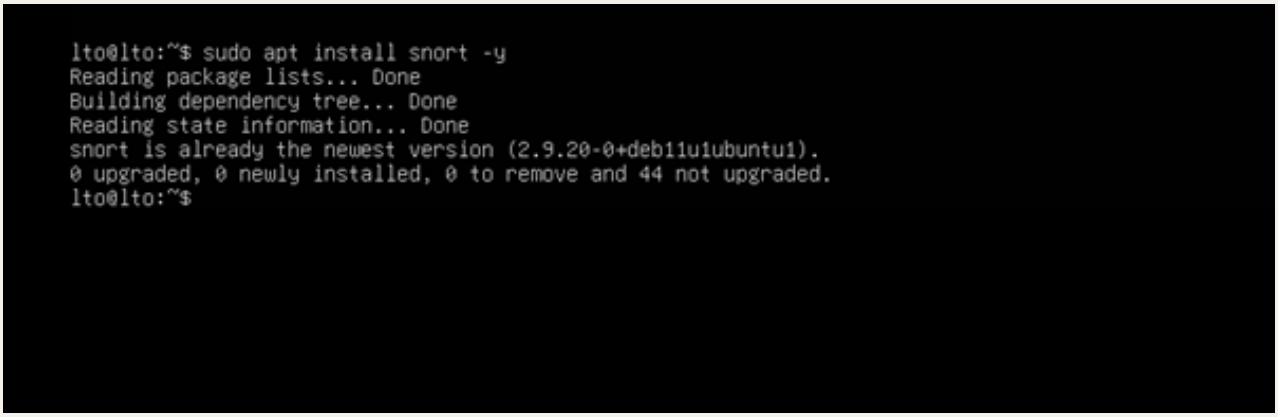

Figure 11: Snort installation process on Ubuntu Server showing package installation and version verification

Custom Detection Rule Configuration

Configuration: A custom detection rule was created to specifically alert on traffic targeting the `/doc` directory. The rule was added to the local rules file using `sudo nano /etc/snort/rules/local.rules`:

Custom Rule:

```
alert tcp any any -> any 80 (msg:"IDS ALERT: CVE-1999-0678 /doc Detected"; content:"/doc/"; sid:1000001; rev:1;)
```

This rule monitors TCP traffic on port 80 and triggers an alert when the `/doc/` path is detected in the request, specifically targeting the CVE-1999-0678 vulnerability.

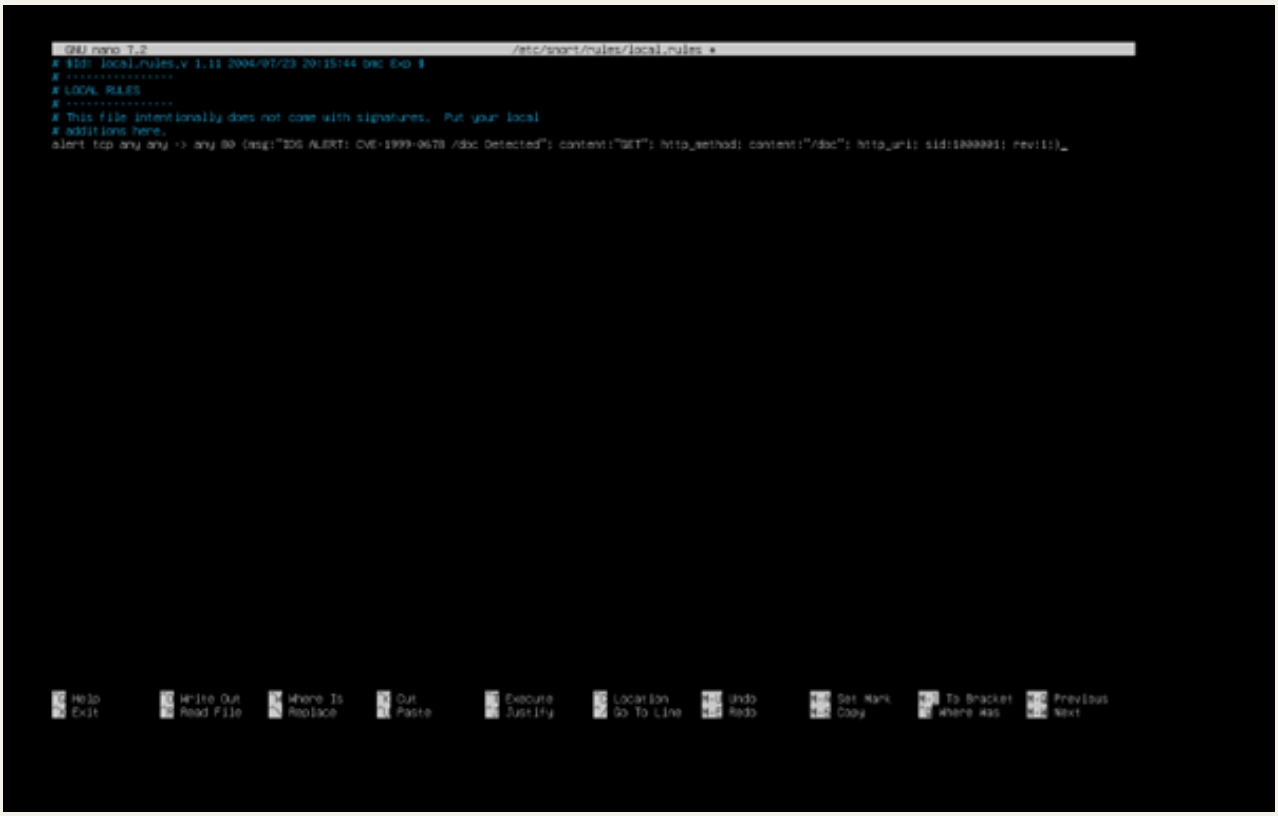

Figure 12: Custom Snort rule configuration in /etc/snort/rules/local.rules file showing the CVE-1999-0678 detection rule

Initial Snort Execution and Troubleshooting

Before starting Snort, the IP address was rechecked to ensure proper network interface configuration. Snort was then started in console mode to observe alerts in real-time.

```
lto@lto:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
       valid_lft forever preferred_lft forever
2: ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:e1:55:b4 brd ff:ff:ff:ff:ff:ff
    altname enp2s1
    inet 192.168.45.132/24 metric 100 brd 192.168.45.255 scope global dynamic ens33
       valid_lft 1052sec preferred_lft 1052sec
    inet6 fe80::20c:29ff:fee1:55b4/64 scope link
       valid_lft forever preferred_lft forever
lto@lto:~$ sudo snort -A console -q -c /etc/snort/snort.conf -i ens33
```

Figure 13: IP address verification and initial Snort execution command on Ubuntu Server

Initial Execution: Snort was started in console mode using:

```
sudo snort -A console -c /etc/snort/snort.conf -i ens33
```

Troubleshooting: Initial tests were obscured by background UPnP network noise. This made it difficult to identify legitimate alerts related to the vulnerability testing.

```
lto@lto:~$ sudo snort -A console -q -c /etc/snort/snort.conf -i ens33
12/14-08:00:16.427656  [**] [1:1917:6] SCAN UPnP service discover attempt [**] [Classification: Detection of a Network Scan] [Priority: 3] {UDP} 192.168.45.1:58
420 -> 239.255.255.250:1900
12/14-08:00:17.442013  [**] [1:1917:6] SCAN UPnP service discover attempt [**] [Classification: Detection of a Network Scan] [Priority: 3] {UDP} 192.168.45.1:58
420 -> 239.255.255.250:1900
12/14-08:00:18.452404  [**] [1:1917:6] SCAN UPnP service discover attempt [**] [Classification: Detection of a Network Scan] [Priority: 3] {UDP} 192.168.45.1:58
420 -> 239.255.255.250:1900
12/14-08:00:19.467000  [**] [1:1917:6] SCAN UPnP service discover attempt [**] [Classification: Detection of a Network Scan] [Priority: 3] {UDP} 192.168.45.1:58
420 -> 239.255.255.250:1900
```

Figure 14: Snort output showing excessive background network noise from UPnP service discovery attempts

Resolution: To resolve this, all existing Snort processes were terminated using `killall snort`, and Snort was restarted with a specific traffic filter: `tcp port 80`. This filter ensured that Snort only monitored HTTP traffic on port 80, eliminating the UPnP noise and allowing clear visibility of web-based attacks.

```
sudo snort -A console -c /etc/snort/snort.conf -i ens33 tcp port 80
```

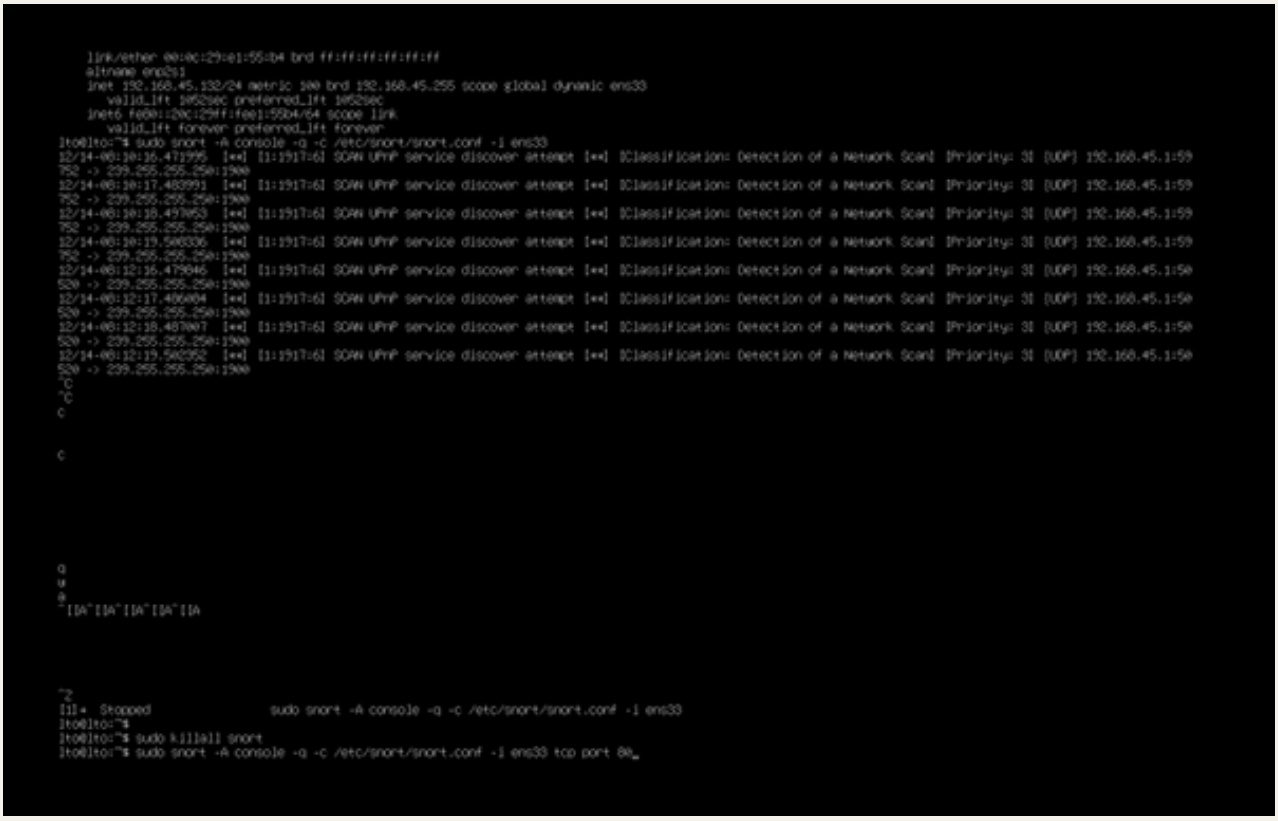

Figure 15: Snort restart with port 80 traffic filter applied, showing termination of previous process and filtered execution

Validation Testing

With Snort running in filtered mode, the vulnerability was exploited again from the Kali Linux machine to verify that the IDS would detect the attack.

Test Execution: The `/doc` directory was browsable, so access was attempted through both browser and command-line methods while monitoring the Snort console on Ubuntu.

Test 1 - Browser Access: Firefox on Kali was used to access `http://192.168.45.132/doc/` while monitoring the Snort console on Ubuntu. The `/doc` directory was browsable, confirming the vulnerability was active.

Test 2 - Command-Line Access: The `curl` command was executed from Kali to generate additional traffic for testing.

Result: Snort successfully triggered the custom alert: “IDS ALERT: CVE-1999-0678 /doc Detected”

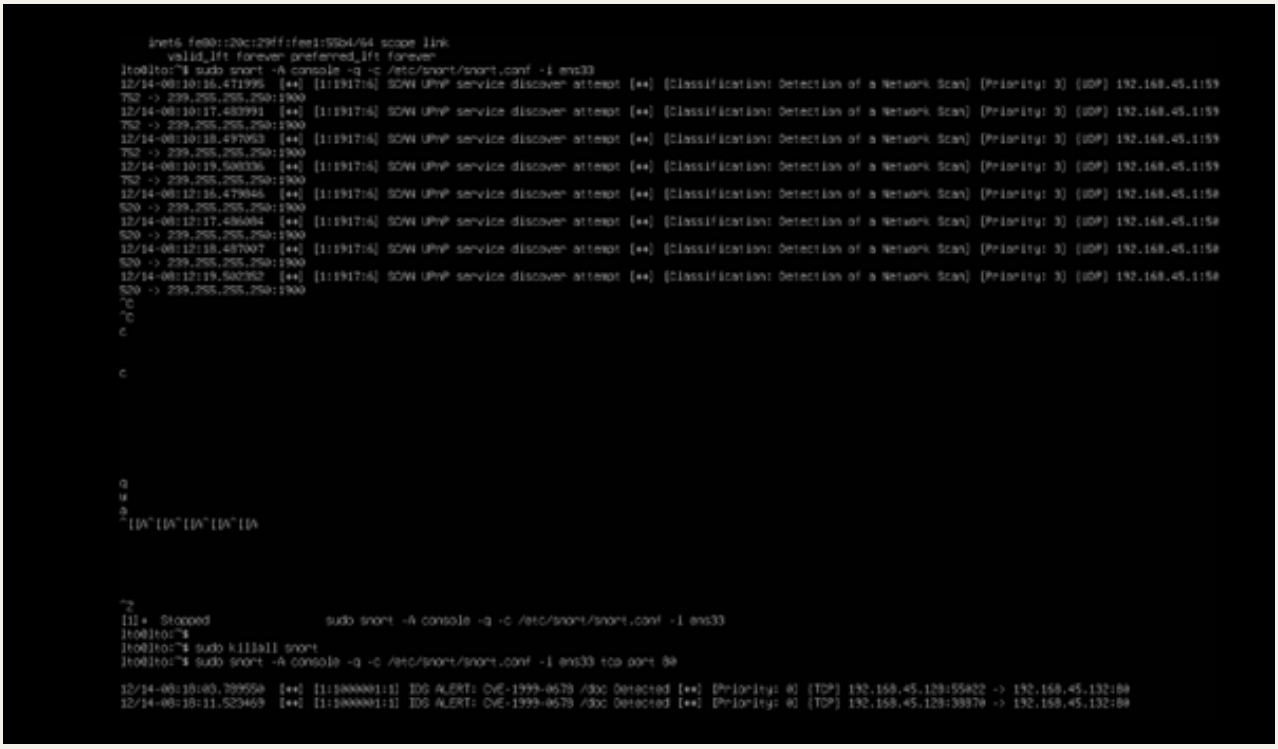

Figure 16: Ubuntu Terminal showing the Snort IDS Alert: “IDS ALERT: CVE-1999-0678 /doc Detected” with source and destination IP addresses

IDS Effectiveness

The implementation successfully demonstrated that Snort could detect exploitation attempts in real-time. The custom rule effectively identified traffic targeting the vulnerable directory, providing immediate notification of potential security breaches.

Phase 5: Remediation and Verification

To permanently fix the security flaw, root-cause remediation was performed on the web server configuration.

Firewall Rule Cleanup

Action: To ensure testing was performed against the actual server configuration rather than firewall rules, all temporary firewall rules were flushed using `sudo iptables -F`:

```
sudo iptables -F
```

This ensured that any security measures were implemented at the application level rather than relying on network-level filtering.

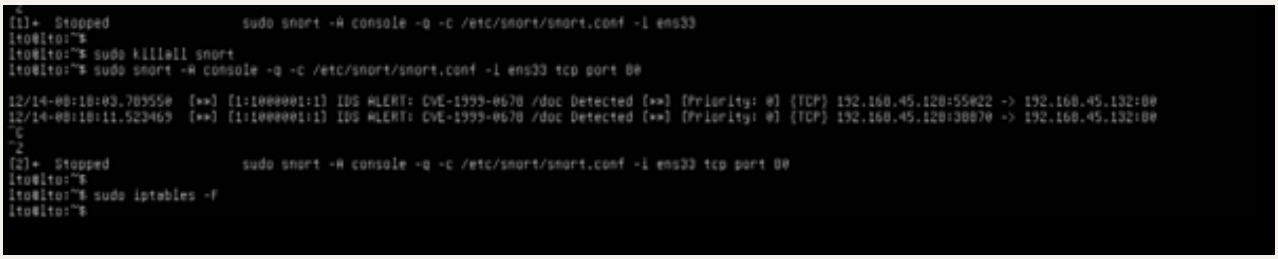

Figure 17: Flushing firewall rules using iptables -F to test application-level security configuration

Vulnerability Remediation

Remediation: The root cause of the vulnerability was addressed by editing `/etc/apache2/sites-available/000-default.conf` and removing the insecure Alias `/doc` configuration block by commenting it out with #.

Original Configuration (Vulnerable):

```
Alias /doc "/usr/share/doc/"
<Directory "/usr/share/doc/">
    Options +Indexes +MultiViews +FollowSymLinks
    AllowOverride None
    Require all granted
</Directory>
```

Remediated Configuration: The configuration block was commented out using # to disable the insecure directory alias:

```
# Alias /doc "/usr/share/doc/"
# <Directory "/usr/share/doc/">
#     Options +Indexes +MultiViews +FollowSymLinks
#     AllowOverride None
#     Require all granted
# </Directory>
```

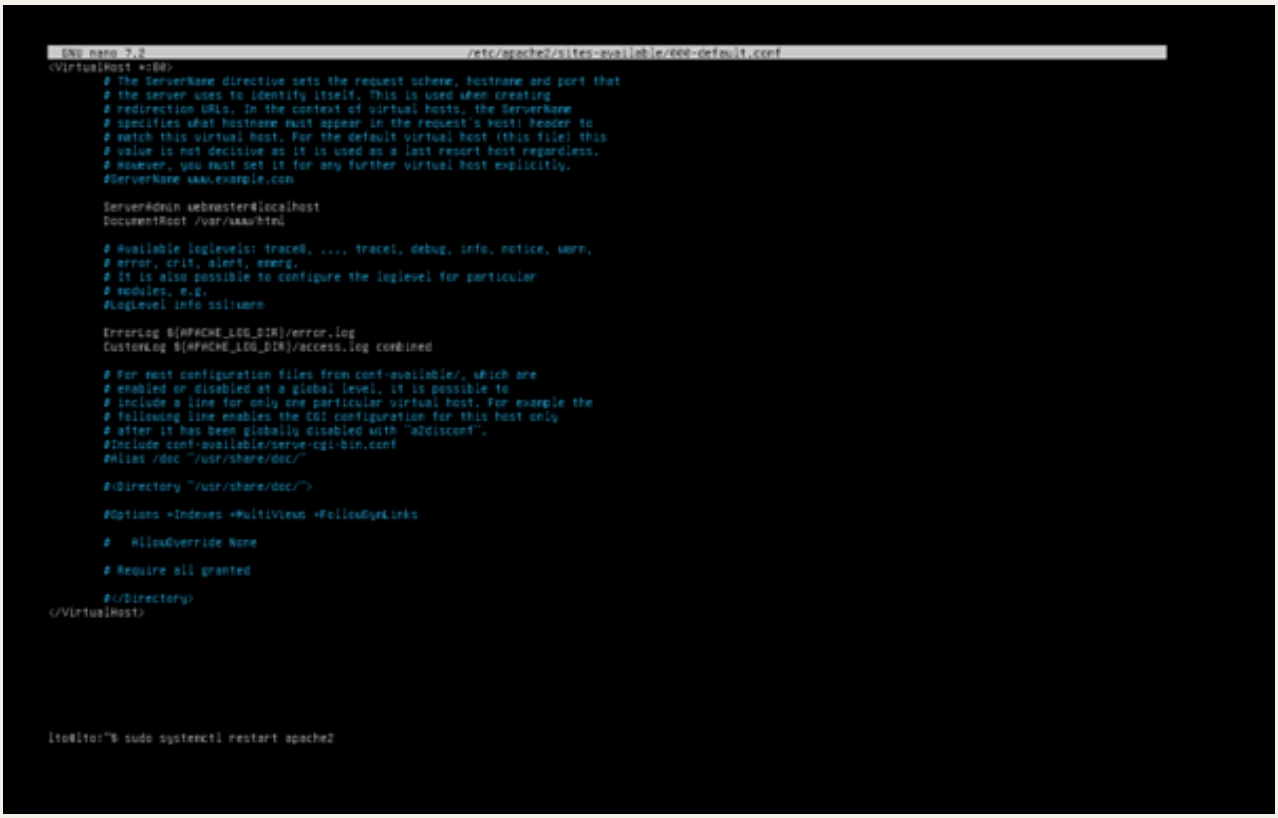

Figure 18: Remediated Apache configuration with vulnerable /doc alias section commented out using # symbols, followed by Apache service restart

Post-Remediation Verification

Final Verification: Access to the `/doc` directory was attempted again from the Kali machine to verify successful remediation.

Test 1 - Browser Access: Firefox on Kali was used to attempt access to `http://192.168.45.132/doc/`.

Result: The page failed to load, indicating that the directory was no longer accessible. The browser displayed a “Not Found” error page.

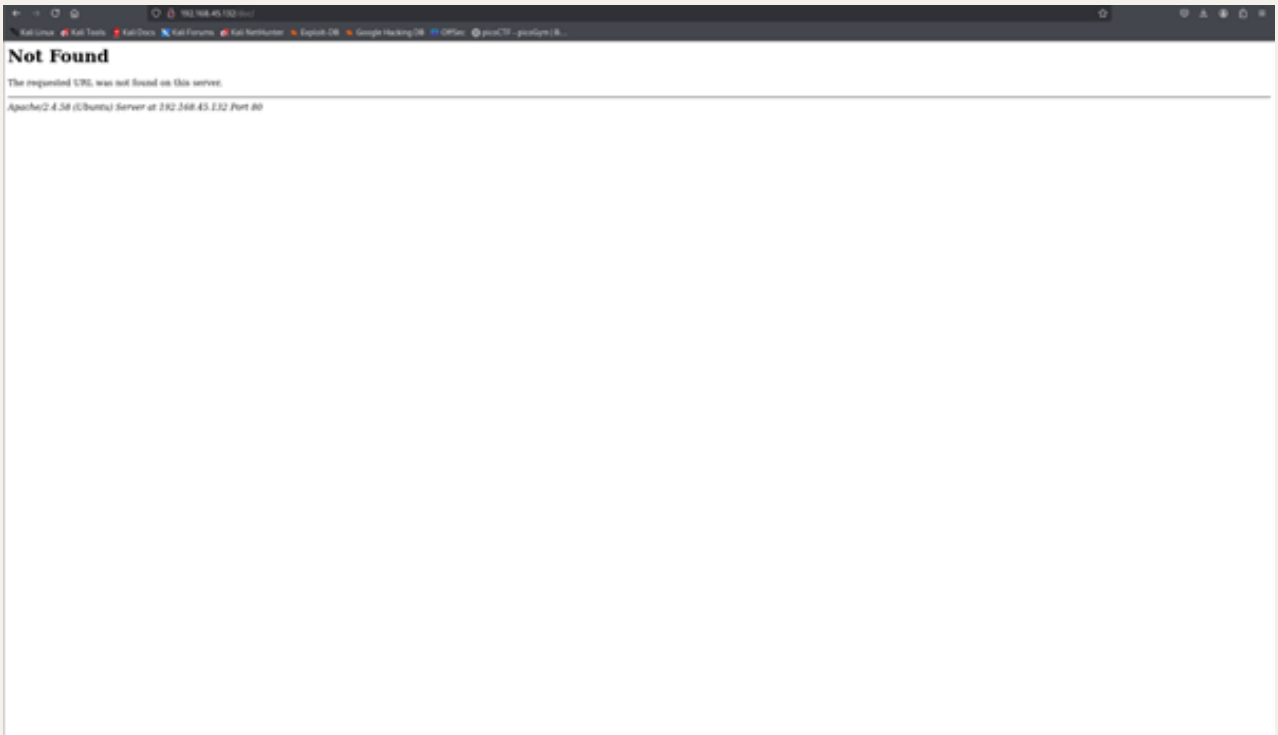

Figure 19: Firefox browser showing “Not Found” error page when attempting to access http://192.168.45.132/doc/ after remediation

Test 2 - Command-Line Verification: The `curl` command was executed to check the HTTP response:

```
curl -I http://192.168.45.132/doc/
```

Result: The command returned `404 Not Found`, confirming that the directory alias had been successfully removed and the vulnerability was no longer exploitable.

Before Remediation:

```
HTTP/1.1 200 OK
```

After Remediation:

```
HTTP/1.1 404 Not Found
```

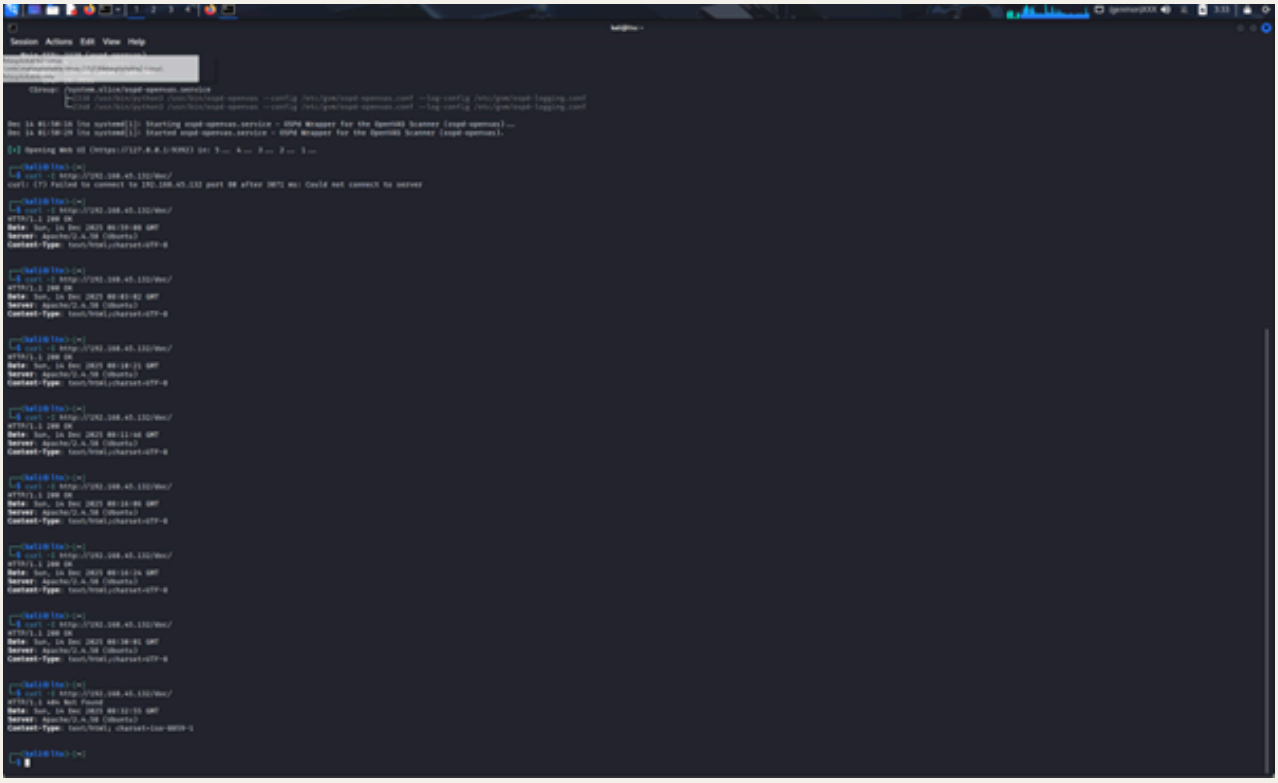

Figure 20: curl command output showing HTTP/1.1 404 Not Found response after remediation, confirming successful vulnerability removal

Remediation Confirmation

The change in HTTP response status from `200 OK` to `404 Not Found` definitively confirmed that:

1. The vulnerability had been successfully remediated
2. The `/doc` directory was no longer accessible
3. The server was no longer susceptible to the CVE-1999-0678 directory traversal attack
4. The remediation was effective at the application configuration level

Discussion

Worst-Case Scenario and Response Strategy

In the worst-case scenario, if the CVE-1999-0678 vulnerability were left unmitigated in a production environment, an attacker could continuously access exposed documentation directories on the web server. This information disclosure could reveal detailed information about installed software packages, versions, directory structures, and configuration examples. Such information could significantly reduce the effort required for attackers to plan more advanced attacks, including targeting known vulnerabilities in specific software versions, conducting service-specific exploits, or preparing privilege escalation attempts.

In a more severe case, the exposed documentation might include sample configuration files or references to sensitive paths, which could further assist attackers in identifying weak access controls or misconfigurations. Over time, this could lead to a chain attack, where initial information disclosure becomes the entry point for more damaging compromises such as unauthorized access, data leakage, or service disruption.

To handle this worst-case situation, immediate response actions should include isolating the affected system from untrusted networks, disabling the vulnerable service or configuration, and conducting a full configuration audit of the web server. Logs from the IDS should be reviewed to determine the scope and duration of exploitation attempts. If evidence of broader compromise

is found, incident response procedures such as system restoration from trusted backups and credential rotation should be initiated to fully contain the threat.

Avoiding Similar Threats in the Future

To prevent similar vulnerabilities in the future, organizations should adopt a proactive and layered security approach. First, secure-by-default configurations should be enforced on web servers, ensuring that directory listing, unnecessary aliases, and access to internal documentation are disabled unless explicitly required. Regular reviews of web server configuration files can help identify and remove insecure or legacy settings before they are exploited.

Second, continuous vulnerability management practices should be implemented. Automated vulnerability scanning tools such as GVM should be used regularly to identify exposed services and potential misconfigurations. However, automated scanning should be supplemented with manual verification and configuration reviews, as some vulnerabilities—especially configuration-based issues—may not be accurately classified by automated tools.

Third, deploying and maintaining intrusion detection or prevention systems is essential for early detection of suspicious activity. Custom IDS rules, such as those implemented in this project, allow security teams to detect specific attack patterns relevant to their environment. Proper tuning of IDS systems, including traffic filtering and rule optimization, helps reduce false positives and improves alert reliability.

Finally, security awareness and administrative best practices play a critical role. System administrators should follow the principle of least privilege, regularly apply updates, and document configuration changes. By combining secure configurations, continuous monitoring, and proper incident response planning, organizations can significantly reduce the likelihood and impact of similar information disclosure vulnerabilities.

Summary and Conclusions

This project successfully demonstrated a complete security testing and remediation workflow for the CVE-1999-0678 vulnerability:

1. **Network Setup:** Established a controlled test environment with proper network connectivity between attacker and victim machines
2. **Vulnerability Introduction:** Intentionally introduced the CVE-1999-0678 vulnerability for testing purposes
3. **Automated Scanning:** Deployed GVM to perform automated vulnerability scanning, identifying the web server service
4. **Manual Verification:** Performed manual testing to confirm the vulnerability was active and exploitable
5. **IDS Implementation:** Successfully deployed Snort with custom detection rules to identify exploitation attempts
6. **Real-Time Detection:** Validated that the IDS could detect attacks in real-time with proper traffic filtering
7. **Remediation:** Permanently fixed the vulnerability by modifying the server configuration
8. **Verification:** Confirmed successful remediation through comprehensive testing